Module Guide for DAIIM

Team 6,
Robert Lam
Nathan Hum
Aidan Lao
Keira Laskoski

November 13, 2026

1 Revision History

Date	Version	Notes
November 13, 2026	0.1	Updated the system overview, likely and unlikely changes, and module decomposition sections

2 Reference Material

This section records information for easy reference.

2.1 Abbreviations and Acronyms

symbol	description
AC	Anticipated Change
DAG	Directed Acyclic Graph
M	Module
MG	Module Guide
OS	Operating System
R	Requirement
SC	Scientific Computing
SRS	Software Requirements Specification
DAIIM	Explanation of program name
UC	Unlikely Change
[etc. —SS]	[... —SS]

Contents

1 Revision History **i**

2 Reference Material **ii**
2.1 Abbreviations and Acronyms ii

3 Introduction **1**

4 Anticipated and Unlikely Changes **2**
4.1 Anticipated Changes 2
4.2 Unlikely Changes 3

5 Module Hierarchy **4**

6 Connection Between Requirements and Design **5**

7 Module Decomposition **7**
7.1 Hardware Hiding Modules 7
7.2 Behaviour-Hiding Module 8
7.2.1 Provider Registry (M1) 8
7.2.2 Marketplace Action Preflight (M2) 8
7.2.3 Request Lifecycle (M3) 8
7.3 Software Decision Module 9
7.3.1 Transaction Manager (M18) 9

8 Traceability Matrix **9**

9 Use Hierarchy Between Modules **11**

10 User Interfaces **11**

11 Design of Communication Protocols **12**

12 Database Design **12**

13 Timeline **12**

14 Generative AI Use Disclosure **12**
14.1 AI Tools Used 12
14.2 How and Where Used 12
14.3 Verification of AI-Generated Content 12

List of Tables

1 Module Hierarchy 5
2 Trace Between Requirements and Modules 10
3 Trace Between Likely Changes and Modules 10
4 Trace Between Unlikely Changes and Modules 11

List of Figures

1 Use hierarchy among modules 11

3 Introduction

Decomposing a system into modules is a commonly accepted approach to developing software. A module is a work assignment for a programmer or programming team (**?**). We advocate a decomposition based on the principle of information hiding (**?**). This principle supports design for change, because the "secrets" that each module hides represent likely future changes. Design for change is particularly valuable in software development, where modifications are frequent, especially during initial development as the solution space is explored. The system described in this document is the Decentralized Artificial Intelligence Inference Marketplace (DAIIM), a web-based platform where users define inference tasks, submit requests, and receive results aggregated from independent providers. An on-chain program records tasks and provider registrations, governs each request's acceptance and commit–reveal phases, and applies the rules for aggregation, stakes, penalties, reputation, and rewards. Off-chain components support user and provider interactions by managing profiles and wallets, preparing transactions, storing and protecting request inputs, and reading on-chain state. Our design follows the rules laid out by **?**, as follows:

- System details that are likely to change independently should be the secrets of separate modules.
- Each data structure is implemented in only one module.
- Any other program that requires information stored in a module's data structures must obtain it by calling access programs belonging to that module.

After completing the first stage of the design, the Software Requirements Specification (SRS), the Module Guide (MG) is developed (**?**). The MG specifies the modular structure of the system and is intended to allow both designers and maintainers to easily identify the parts of the software. The potential readers of this document are as follows:

- New project members: This document provides an overview of the system's modular structure, allowing new members to understand the overall design and quickly locate the modules relevant to their work.
- Maintainers: The hierarchical structure of the module guide helps maintainers understand the responsibilities and relationships between modules when modifications are required. Maintainers should update the relevant sections of this document when changes are made to the system.
- Designers: The module guide can be used to evaluate the consistency, feasibility, and flexibility of the system's decomposition. Designers can use it to verify that module responsibilities are appropriately separated and that anticipated changes can be accommodated without requiring unnecessary modifications to other modules.

The rest of the document is organized as follows. Section 4 lists the anticipated and unlikely changes to the system. Section 5 summarizes the module decomposition constructed

according to the information-hiding principle. Section 6 specifies the connections between the software requirements and the modules. Section 7 gives a detailed description of the modules. Section 8 includes two traceability matrices. One checks the completeness of the design against the requirements provided in the SRS. The other shows the relationship between anticipated changes and the modules. Section 9 describes the use relations between modules.

4 Anticipated and Unlikely Changes

This section identifies design decisions that may change as DAIIM develops and assumptions that are expected to remain the same. Anticipated changes are listed in Section 4.1, and unlikely changes are listed in Section 4.2.

4.1 Anticipated Changes

Anticipated changes are changes to requirements, design decisions, or implementation choices that the modules are intended to accommodate. Each decision should be hidden by the module responsible for it so that changing the decision has limited impact on other modules. Changes to values during operation, such as a provider joining a task or a request entering a new phase, are not anticipated changes in this sense.

AC1: The on-chain representation and validity rules for task definitions and the token-curated provider registry, including per-task reputation and how providers qualify for and remain listed under each task, may change.

AC2: The preliminary checks and error feedback for proposed marketplace actions, such as creating tasks or requests and registering providers, may change.

AC3: The lifecycle and control flow of an inference request, including the phases, deadlines, commitment-reveal rules, and cancellation conditions may be revised.

AC4: The algorithm used to combine provider predictions may change.

AC5: The decision making process for reward eligibility and the calculation of each eligible provider's share of rewards for an inference request may change.

AC6: The conditions and amounts for slashing provider stake may change as the system's incentives are evaluated.

AC7: The formula for deciding how much a provider's reputation should change after an inference request may be updated.

AC8: The mechanisms used for token transfers, minting, and burning may change.

AC9: Supported cryptographic wallet implementations and the methods used to connect wallets, verify ownership, and request signatures may change.

AC10: The storage and retrieval of user profiles including verified wallet addresses, preferences, and user history may change.

AC11: The mechanism for reading on-chain blockchain data and detecting their changes may change.

AC12: The way blockchain transactions are constructed, simulated, checked for sufficient funds, sent, and confirmed may change.

AC13: The storage, encryption, access control, and deletion mechanisms for off-chain request input data may change.

AC14: The method by which provider clients parse the blockchain data for relevant information such as new inference requests may change.

AC15: The procedure for establishing a provider client's blockchain connection and access to the provider's wallet may change.

AC16: The way provider-owned models are invoked and their predictions are prepared and submitted to the on-chain program may change.

AC17: The implementation of a provider-owned inference model may change while it continues to accept the required task input and produce a conforming prediction.

AC18: The user interface through which requesters create tasks, submit inference requests, and inspect request status, results, and history may change.

AC19: The user interface through which providers register for tasks, inspect their registrations, reputation, stakes, and rewards may change.

4.2 Unlikely Changes

Although the system should accommodate anticipated changes, certain fundamental design decisions are not expected to change during the scope of the project. These decisions establish the core architecture and purpose of DAIIM, and changing them would likely require modifications across multiple parts of the system.

UC1: DAIIM uses Solana as the underlying public blockchain. It is used to execute marketplace rules and maintain authoritative records of the protocol's state. Replacing this with a different blockchain network or an off-chain authority would require a very different protocol architecture.

UC2: Inference requests refer to defined tasks, and providers participate on a per-task basis. Removing tasks or making provider participation independent of tasks would change the registry, request protocol, and user workflows.

UC3: Each successful inference result is based on predictions from multiple independent providers. A single-provider service would change the purpose of aggregation, reputation, and provider incentives.

UC4: Requester payments, provider stakes, penalties, and rewards remain token-based. Removing tokens would change the marketplace's participation and settlement mechanisms.

5 Module Hierarchy

This section provides an overview of the module design. Table 1 groups the leaf modules by the decisions they hide. The Model is implemented by inference providers rather than by DAIIM.

M1: Provider Registry

M2: Marketplace Action Preflight

M3: Request Lifecycle

M4: Prediction Aggregator

M5: Rewards Calculator

M6: Penalty Enforcer

M7: Reputation Handler

M8: Request Input Manager

M9: Requester Interface

M10: Provider Interface

M11: Request Listener

M12: Participation Client

M13: Model (provided externally)

M14: Token Manager

M15: Wallet Service

M16: User Information

M17: Chain Observer

M18: Transaction Manager

M19: Provider Setup

Table 1: Module Hierarchy

Category	**Module**
Hardware-Hiding	No project-specific hardware-hiding module
Behaviour-Hiding	M1: Provider Registry M2: Marketplace Action Preflight M3: Request Lifecycle M4: Prediction Aggregator M5: Rewards Calculator M6: Penalty Enforcer M7: Reputation Handler M8: Request Input Manager M9: Requester Interface M10: Provider Interface
Software Decision	M14: Token Manager M15: Wallet Service M16: User Information M17: Chain Observer M18: Transaction Manager M19: Provider Setup M11: Request Listener M12: Participation Client M13: Model (provided externally)

6 Connection Between Requirements and Design

The design of DAIIM is intended to satisfy the requirements developed in the SRS. In this stage, the system is decomposed into modules. The connection between requirements and modules is listed in Table 2.

- **Wallet Manager (M15):** To satisfy FR1 and FR4, wallet authentication and transaction signing are handled through a dedicated wallet module. This module verifies ownership of wallet addresses and provides the interface for users to authorize transactions without exposing their private keys to DAIIM.

- **Chain Observer (M17):** To satisfy FR58, FR56, and FR57, a dedicated module monitors blockchain activity and retrieves transaction and account information. This separates reading and interpreting blockchain data from the modules responsible for initiating transactions and managing marketplace operations.

- **Request Input Manager (M8):** To satisfy FR25, FR59, FR60, and FR61, a dedicated module manages the secure storage, encryption, access, and deletion of inference request inputs. Input data is kept off-chain and made available only to the requester and providers that accepted the corresponding request.

- **User Interface (M??):** To satisfy FR3, FR10, FR5, and FR36, the web interface provides users with access to marketplace information, task creation, inference requests, and aggregated results. It also presents transaction information and marketplace activity in a way that does not require users to interact directly with the blockchain.

- **User Information (M16):** To support FR1 and FR14, a dedicated module manages user profile information associated with verified wallet addresses. This allows user-specific information, such as provider activity and preferences, to be maintained separately from the underlying blockchain records.

- **Provider Setup (M19):** To support FR12 and FR14, a dedicated module handles the provider's initial connection to the blockchain and retrieves the provider's registration information for available tasks. This separates provider initialization and configuration from the ongoing handling of inference requests.

- **Request Listener (M11):** To satisfy FR20 and FR21, a dedicated module monitors inference requests that are relevant to a provider's registered tasks. It identifies new requests and communicates their details, payment information, and applicable deadlines to the provider.

- **Participation Client (M12):** To satisfy FR25, FR27, FR28, and FR29, a dedicated client module handles the provider's participation in inference requests. It retrieves authorized input data, generates the prediction commitment using a cryptographically secure salt, and submits commitments and reveals directly to the on-chain program.

- **Provider Registry (M1):** To satisfy FR12, FR13, and FR51, a dedicated module manages provider registration for individual inference tasks. It maintains provider eligibility, registration status, and task-specific staking requirements so that only eligible providers can participate in requests.

- **Request Lifecycle (M3):** To satisfy FR17 through FR31, a dedicated module manages the state and progression of inference requests. The request lifecycle is implemented as a state machine that controls the acceptance, commit, and reveal phases, enforces their deadlines, validates provider participation, and determines whether a request can proceed or must be cancelled.

- **Settlement (M??):** To satisfy FR35, FR39, FR40, and FR41, a dedicated module handles the finalization of inference requests. It calculates the aggregated result using the reputation-weighted median algorithm, updates provider reputations using the confidence-based reputation algorithm, and determines the distribution of rewards based on the resulting provider reputations.

- **Token Manager (M14):** To satisfy FR19, FR32, FR33, and FR47, a dedicated module manages token-related operations. These include transferring payments into escrow, managing provider stakes, releasing reserved funds, distributing tokens, and applying penalties such as slashing or burning when required.

- **Transaction Manager (M18):** To satisfy FR55, FR56, and FR57, a dedicated module coordinates the preparation and submission of user-authorized transactions. It validates transaction parameters, checks available balances, simulates transactions before submission, and tracks their status to prevent duplicate submissions or incorrect reporting of transaction outcomes.

7 Module Decomposition

Modules are decomposed according to the principle of "information hiding" proposed by **?**. The *Secrets* field in a module decomposition is a brief statement of the design decision hidden by the module. The *Services* field specifies *what* the module will do without documenting *how* to do it. For each module, a suggestion for the implementing software is given under the *Implemented By* title. If the entry is *OS*, this means that the module is provided by the operating system or by standard programming language libraries. *DAIIM* means the module will be implemented by the DAIIM software.

Only the leaf modules in the hierarchy have to be implemented. If a dash (–) is shown, this means that the module is not a leaf and will not have to be implemented.

7.1 Hardware Hiding Modules

Secrets: The data structure and algorithm used to implement the virtual hardware.

Services: Serves as a virtual hardware used by the rest of the system. This module provides the interface between the hardware and the software. So, the system can use it to display outputs or to accept inputs.

Implemented By: OS

7.2 Behaviour-Hiding Module

Secrets: The contents of the required behaviours.

Services: Includes programs that provide externally visible behaviour of the system as specified in the software requirements specification (SRS) documents. This module serves as a communication layer between the hardware-hiding module and the software decision module. The programs in this module will need to change if there are changes in the SRS.

Implemented By: –

7.2.1 Provider Registry (M1)

Secrets: The representation of inference tasks and the token-curated provider registry for each task. Owns the on-chain validity rules for task definitions and the rules for admitting, ordering, and removing providers from the registry.

Services: Records and retrieves on-chain task definitions and per-task provider registrations, reputation, and stake. Rejects task definitions that violate protocol constraints and determines whether a provider is eligible to join or remain in a task's registry.

Implemented By: DAIIM on-chain program

Type of Module: Abstract Object

7.2.2 Marketplace Action Preflight (M2)

Secrets: Which preliminary checks are performed before transaction submission and how errors are reported to users.

Services: Checks proposed tasks, requests, and provider actions and reports actionable errors before invoking the on-chain program. The on-chain program is authoritative on protocol-critical rules.

Implemented By: DAIIM off-chain application

Type of Module: Library

7.2.3 Request Lifecycle (M3)

Secrets: The representation of inference requests and accepted-provider records, the request state machine and deadlines, and the commitment and reveal validation rules.

Services: Creates inference requests, records provider acceptances, commitments, and reveals, advances request phases, rejects invalid participation, and determines when a request must be cancelled.

Implemented By: DAIIM on-chain program

Type of Module: Abstract Data Type

7.3 Software Decision Module

Secrets: The design decision based on mathematical theorems, physical facts, or programming considerations. The secrets of this module are *not* described in the SRS.

Services: Includes data structure and algorithms used in the system that do not provide direct interaction with the user.

Implemented By: –

7.3.1 Transaction Manager (M18)

Secrets: How blockchain transactions are constructed, simulated, checked for sufficient funds, submitted, and confirmed.

Services: Constructs transactions for marketplace actions, simulates them, checks balances and fees, obtains wallet authorization, submits signed transactions, and tracks their status to avoid duplicate submissions.

Implemented By: DAIIM off-chain application

Type of Module: Abstract Object

8 Traceability Matrix

This section shows the traceability between the system modules and the requirements, as well as between the modules and the likely and unlikely changes anticipated for DAIIM. These matrices demonstrate which modules are responsible for satisfying the requirements and which modules may need to be modified if the system evolves.

Table 2: Trace Between Requirements and Modules

Requirement(s)	**Modules**
FR1, FR2, FR4	M15, M16, M??, M18
FR3, FR10	M??, M1, M16
FR5–FR9	M??, M3, M18, M14
FR11, FR12, FR13, FR51	M1, M14, M18, M15, M19
FR14	M??, M16, M1, M17
FR17, FR18, FR19, FR20	M??, M3, M8, M14, M18, M11
FR21–FR24, FR23	M3, M1, M11, M14
FR25, FR59, FR60, FR61	M8, M12, M3
FR26, FR15–FR16, FR27–FR29	M3, M12, M8, M11
FR30, FR31, FR32	M3, M1, M14, M12
FR35, FR36, FR37, FR38	M??, M3, M??, M17
FR39, FR40, FR41	M??, M1, M14
FR42–FR46, FR34	M??, M3, M14
FR47–FR53	M1, M??, M14, M3
FR55, FR56, FR57, FR58	M18, M15, M17, M??, M14
SAFR.SA.1–SAFR.SA.6, SAFR.SA.7, SAFR.SA.8, SAFR.SA.9	M18, M15, M14, M??, M??
NFR9, NFR10	M8, M12
NFR11, NFR12	M12, M3
NFR1, NFR2	M??, M14
NFR14, NFR16	M3, M??, M1, M14
NFR6, NFR7, NFR5	M??, M18, M17
NFR4, NFR13, NFR3, NFR17	M??, M15, M16

Table 3: Trace Between Likely Changes and Modules

Likely Change (LC)	**Modules**
LC1	M19, M12, M3, M??
LC2	M1, M3, M??, M14, M11
LC3	M??, M1, M14
LC4	M11, M1, M3
LC5	M??, M16

Table 4: Trace Between Unlikely Changes and Modules

Unlikely Change (LC)	**Modules**
LC6	M??, M1, M3, M??, M12
LC7	M??, M15, M19

9 Use Hierarchy Between Modules

In this section, the uses hierarchy between modules is provided. A module A uses a module B if the correct execution of A depends upon the availability of a correct implementation of B. The arrows in the uses hierarchy are directed from the module that requires a service to the module that provides that service. For DAIIM, the uses hierarchy is organized around the User Interface, the provider-side SDK modules, the supporting off-chain modules, and the core on-chain modules. The User Interface (M??) depends on the Wallet Manager (M15), User Information (M16), Transaction Manager (M18), and Chain Observer (M17) to support wallet authentication, user information retrieval, transaction submission, and the display of blockchain state. On the provider side, the Provider Setup module (M19) uses the Wallet Manager (M15), User Information (M16), and Chain Observer (M17) to establish provider identity and retrieve registration information. The Request Listener (M11) uses the Chain Observer (M17) to monitor relevant requests and blockchain events. The Participation Client (M12) uses the Provider Setup module (M19), Request Input Manager (M8), and Transaction Manager (M18) to retrieve authorized request inputs and participate in inference tasks. The Transaction Manager (M18) uses the Wallet Manager (M15) and Chain Observer (M17) to prepare, authorize, and verify transactions. Marketplace Action Preflight (M2) uses the Transaction Manager (M18) when a preliminary check requires transaction simulation. The core on-chain modules provide the functionality required to operate the marketplace. The Request Lifecycle module (M3) uses the Provider Registry (M1) and Token Manager (M14) to manage provider eligibility and request-related token operations. The Settlement module (M??) uses the Request Lifecycle module (M3), Provider Registry (M1), and Token Manager (M14) to calculate final results, update provider reputations, and distribute rewards or penalties. Figure 1 illustrates the uses relationships between the modules. The resulting hierarchy is a directed acyclic graph (DAG), with higher-level modules depending on lower-level modules that provide the services they require.

Figure 1: Use hierarchy among modules

10 User Interfaces

[Design of user interface for software and hardware. Attach an appendix if needed. Drawings, Sketches, Figma —SS]

11 Design of Communication Protocols

[If there is communication between hardware and software, it may be necessary to explain the design of the communication protocol. This section should be filled in with NA if it is not appropriate for a given project. —SS]

12 Database Design

[If a database is a significant component of the project, the database design should be summarized here. This section should be filled in with NA if it is not appropriate for a given project. —SS]

13 Timeline

[Schedule of tasks and who is responsible —SS]

[You can point to GitHub if this information is included there —SS]

14 Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

14.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

14.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

14.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.

- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]